

Senior Backend Architecture Review & Refactor Plan — Football Auction Platform

Paste this whole document to your coding agent as the task brief. It supersedes the earlier, narrower auction-engine-only prompt — this one covers the full product surface now that the complete workflow is visible: auth, dashboard, player database, team pages, profile/settings, live auction, chat, WebRTC, host presence/cancellation, and persistence.

Role

You are acting as a senior backend / distributed-systems engineer reviewing and refactoring a real-time football-player auction platform built with:

- Next.js + TypeScript, Zustand client state, Next.js API routes, MongoDB
- Auth via `scrypt`-hashed passwords, session persisted client-side in `localStorage` + Zustand
- Realtime room sync via HTTP polling (1s snapshot poll, 4s presence heartbeat)
- WebRTC for camera/mic, signaled through the room API
- A single `rooms` collection as the authoritative store for the live auction, with a documented but partially-used normalized schema (`auctions`, `teams`, `players`, `bids`, `auction_history`, `chat_messages`) for future separation

The product spans: signup/login → dashboard → create/join a room → lobby → live bidding → chat → sold/unsold settlement → results, plus supporting player-database and profile/team browsing pages.

What's Confirmed Working

- Auth: signup/login route through `authStore` → `authService` → API → `userStore` → MongoDB, with server-side validation and `scrypt` hashing. Session guarded by `AuthGuard`.
- Room lifecycle scaffolding: `roomStore.create()`, `.join()`, `.start()` persist correctly and set `lobby` → `live` status.
- `roomStore` already claims **revision-aware persistence** intended to stop concurrent requests from silently overwriting a newer room state, and the timer is described as deriving remaining time from `timerEpoch` + server clock (`liveWithClock()`), not a client counter.
- Chat, history, and results are wired through the same room snapshot/action pipeline.
- WebRTC signaling is routed through a dedicated `signals` API route, separate from the room state (`rooms/[auctionId]/signals`), which is architecturally correct — media should never touch MongoDB.
- Host leave triggers `roomStore.leave()` → auction `cancelled`; a stale host heartbeat also auto-cancels the room; the host's own reads refresh the heartbeat to avoid a false cancellation during their own start action.

Reported / Suspected Problems (carried over from the auction engine, now viewed in full context)

- Live timers not synced between users despite the described `timerEpoch` model — suggests the *design* is right but something in `progressLive()/tick()/polling` cadence isn't honoring it consistently.
- Visible delay between clicking bid and it appearing for others — expected given 1s polling as the primary transport, not just a bug.
- Timer sometimes restarts after a bid when it shouldn't — points to an implicit reset inside the bid-then-`tick()` path rather than an explicit `timerPolicy`.
- Players occasionally marked unsold incorrectly — likely a race between `progressLive()` running from multiple trigger points (client tick, other clients' polling reads, host action) without a single serialized settlement path.
- Multiple clients can each trigger progression — same root cause.
- Host heartbeat/cancellation logic is high-risk: a **temporary** network blip or backgrounded tab could cross the "stale heartbeat" threshold and cancel a *live* auction with real bids and squads already in it. This needs a grace period and clear separation from "host intentionally left."
- Chat and auction state currently share the same polling/snapshot channel — a burst of chat messages could add latency to bid/timer-critical fetches, and vice versa.
- The claimed revision-aware persistence needs to be **verified**, not assumed: confirm every write path (bid, sell, unsold, advance, start, join, leave, chat) actually goes through the same version-checked write, and that nothing writes `status`, `currentBid`, or `timer*` fields outside that path.

Phase 1 — Full System Audit (do this before changing code)

Map every flow below to its concrete files and trace every state mutation, exactly as thoroughly for the "surrounding" product as for the auction core — a bug in host-cancellation or chat contention can look identical to a bidding race from the user's perspective.

Flow	Path
Signup	<code>SignupForm</code> → <code>authStore.signup()</code> → <code>authService</code> → <code>POST /api/auth/signup</code> → <code>userStore.signup()</code> → MongoDB
Login	<code>LoginForm</code> → <code>authStore.login()</code> → <code>authService</code> → <code>POST /api/auth/login</code> → <code>userStore.login()</code> → MongoDB
Session guard	<code>AuthGuard</code> (localStorage hydration, route protection, redirects)
Dashboard load	<code>auctionService.listAuctions()</code> → <code>GET /api/rooms</code> → grouped live/recent
Room creation	create page → <code>auctionService</code> → <code>POST /api/rooms</code> → <code>roomStore.create()</code> → MongoDB
Join	join page → <code>auctionService</code> → room API → <code>roomStore.join()</code> → MongoDB
Lobby sync	<code>AuctionRoomClient</code> → <code>useRoomSync</code> (1s snapshot poll, 4s presence) → room API → <code>roomStore</code>
Start	lobby → <code>auctionService.startAuction()</code> → API → <code>roomStore.start()</code>

Flow	Path
Bid	<code>BidButton</code> → <code>auctionStore</code> → <code>auctionService</code> → API → <code>roomStore</code> → MongoDB
Timer	<code>useAuctionTimer</code> → <code>tick()</code> → API → <code>roomStore.progressLive()</code>
Settlement	sell / timer expiry → <code>roomStore</code> → <code>assignPlayerToTeam()</code> → <code>advanceLot()</code>
Chat	<code>ChatPanel</code> → <code>chatStore</code> → <code>chatService</code> → room API chat action → <code>roomStore</code> → next snapshot fan-out
WebRTC	<code>useRoomMedia</code> → <code>mediaStore</code> → <code>RTCPeerConnection</code> → signals API → remote peer → <code>ParticipantVideoCard</code>
Leave / cancel	<code>AuctionLeaveButton</code> / <code>AuctionControls</code> → <code>roomStore.leave()</code> or cancel action; heartbeat expiry → auto-cancel
Persistence	<code>roomStore</code> (revision, heartbeat, all sub-collections embedded in <code>rooms</code>) → <code>mongodb.ts</code> / <code>schemas.ts</code>

For each flow, identify:

- every place that mutates room state (status, timer, bids, teams, chat, participants)
- every place `progressLive()` / `advanceLot()` / `tick()` is invoked
- how the claimed revision number is actually enforced — is it checked on every write, or only some?
- the exact heartbeat-staleness threshold and whether it distinguishes "host tab backgrounded/network blip" from "host truly gone"
- whether chat writes and auction-state writes contend for the same document/revision, and what happens if they race
- what happens to in-flight WebRTC connections when a room is cancelled or completed

Deliverable: a concise report of what's actually true in the code vs. what the docs claim (especially the revision-number and timer-epoch claims), before any refactor begins.

Phase 2 — Design Principles

A. Auction core (unchanged core rules, now more precisely justified)

1. **Server is sole authority.** Clients send commands (`BID`, `SELL`, `UNSOLD`, `START`, `PAUSE`, `RESUME`, `LEAVE`, `CANCEL`) — never decide status, timer expiry, or settlement themselves.
2. **Explicit state machine:** `LOBBY` → `LIVE` → `AWAITING_SETTLEMENT` → `SOLD_OVERLAY/UNSOLD_OVERLAY` → `LIVE` → `COMPLETED`, with `LIVE` ⇌ `PAUSED` and any state → `CANCELLED`. Reject illegal transitions server-side; centralize the transition function so nothing sets `status` directly elsewhere.
3. **Version-guarded writes, verified everywhere.** Every mutation (bid, sell, unsold, advance, start, join, leave, cancel, **and chat**) goes through one `applyToRoom(auctionId, expectedVersion, mutator)` helper that does a compare-and-swap against MongoDB (`WHERE version = expectedVersion`) and increments version on success. If chat currently bypasses this, that's the likely source of the "unrelated delay/race" symptom.

4. **Timer is timestamp-derived, not decremented.** Keep `timerEpoch/timeRemaining`-style fields but formalize as `{ startedAt, durationMs, pausedAt?, accumulatedPausedMs }`; `remaining = duration - (now - startedAt - pausedDuration)`. Server decides expiry; client only renders, ideally at ~100–250ms ticks reading from a computed server-synced deadline (`serverNow + offset`), not from its own `tick()` triggering settlement.
5. **Explicit timer-reset policy.** Define once, centrally, whether/how a bid extends the timer (`timerPolicy: { resetOnBid, resetDurationMs }`). Nothing else may touch timer fields.
6. **Idempotent, single-writer settlement.** `settleCurrentLot()` must be safe to call redundantly from host action, client-reported expiry, another client's poll, or a room read — first caller wins, everyone else gets the same already-settled result. This directly targets "players unsold incorrectly" and "multiple clients trigger progression."
7. **Idempotency keys on commands.** Client-generated `commandId`; server stores processed IDs and replays the original result for a duplicate (double-click, retry, slow network).
8. **Deterministic player queue.** Explicit ordered queue/bucket index; advance exactly once per settlement; enforce `soldPlayerIds ∩ unsoldPlayerIds = ∅` and `currentPlayerId ∉ soldPlayerIds ∪ unsoldPlayerIds` outside of settlement.

B. Presence, leave, and cancellation (new — this is where "smooth UX" is most at risk)

9. **Separate "host offline" from "auction cancelled."** A live auction with real bids and rosters should not vanish because of a dropped wifi packet. Introduce a grace window (e.g. missed heartbeat for N consecutive intervals, not one) before auto-cancelling, and prefer a `HOST_DISCONNECTED` transient status that still allows the auction to resume if the host reconnects within the window, rather than an immediate `CANCELLED`.
10. **Distinguish intentional leave from heartbeat timeout.** `AuctionLeaveButton` (explicit host action) can cancel immediately with confirmation; heartbeat-based detection should not carry the same immediacy. Log which path triggered a cancellation for support/debugging.
11. **Non-host leave must not corrupt state.** Removing a participant/team assignment mid-auction must go through the same versioned command path — never a direct field removal that could race with a bid on that same team.
12. **Cancellation is terminal and clearly labeled.** Once `CANCELLED`, reject all further commands except read; results/history must still show bids and squads accumulated up to that point (already the stated intent — keep it, but enforce it via the state machine, not by convention).

C. Realtime transport and chat

13. **Move live auction propagation off 1-second polling** onto a realtime channel (WebSocket/SSE/Socket.IO). Auction events (`BID_PLACED`, `PLAYER_SOLD`, `PLAYER_UN SOLD`, `AUCTION_PAUSED/RESUMED/COMPLETED/CANCELLED`, `PARTICIPANT_JOINED/LEFT`) and chat messages should ride the same realtime channel but as **distinct event types**, so a burst of chat traffic can't add latency to bid-critical events and vice versa. Keep polling only for initial hydration and reconnect recovery.
14. **Chat is not auction state.** Store and version chat separately from the auction-critical fields (even if still embedded in the same document for now) so a chat write's revision check can never block or race a bid's revision check. If they must share one document, give chat its own sub-revision instead of overloading the room-wide version.

15. **Reconnection recovery.** Client resumes with `{ auctionId, lastKnownVersion, lastKnownChatSeq }`; server replies with a snapshot or the delta of missed events — for both auction state and chat — so a dropped connection self-heals without a manual refresh.

D. Client architecture

16. **One reducer, one source of truth.** `SERVER event → applyAuctionEvent(event) → Zustand → UI`. No component calls `setCurrentBid`, `setCurrentPlayer`, `setTimer`, `setStatus`, or `chat-append` directly — route everything through the reducer so components can't disagree.
17. **Optimistic UI, server-confirmed truth.** Bid button reacts instantly ("Submitting...", disabled) but the amount shown to everyone else comes from the `BID_PLACED` event, not the optimistic guess. Same pattern for host actions (`START`, `SELL`, `UNSOLD`, `PAUSE`, `RESUME`, `CANCEL`): show "Processing..." locally, apply only the confirmed event.
18. **Server-driven overlays and next-lot timing.** `PLAYER_SOLD/PLAYER_UNSOLD` events carry `overlayDurationMs` and next-lot info; clients render on that schedule and never independently decide to advance.

E. Data model and API

19. Formalize the lot model:

```
currentLot: {  
  playerId: string  
  status: "ACTIVE" | "SETTLING" | "SOLD" | "UNSOLD"  
  currentBid: number  
  highestBidderId?: string  
  startedAt: Date  
  endsAt: Date  
}
```

20. **Typed events**, enough payload to update UI without a re-fetch — include the auction ones above plus `CHAT_MESSAGE`, `HOST_DISCONNECTED`, `HOST_RECONNECTED`, `PARTICIPANT_LEFT`.
21. **Move toward explicit command endpoints** rather than one generic `action`: string, at minimum for the state-mutating auction commands:

```
POST /api/rooms  
POST /api/rooms/:id/join  
POST /api/rooms/:id/commands/start  
POST /api/rooms/:id/commands/bid  
POST /api/rooms/:id/commands/sell  
POST /api/rooms/:id/commands/unsold  
POST /api/rooms/:id/commands/pause  
POST /api/rooms/:id/commands/resume  
POST /api/rooms/:id/commands/cancel  
POST /api/rooms/:id/chat  
GET /api/rooms/:id
```

22. **Consistent responses** with machine-readable codes: `AUCTION_NOT_LIVE`, `AUCTION_PAUSED`, `TIMER_EXPIRED`, `INVALID_BID`, `BUDGET_EXCEEDED`, `SQUAD_FULL`, `NOT_TEAM_OWNER`, `NOT_HOST`, `PLAYER_ALREADY_SETTLED`, `DUPLICATE_COMMAND`, `STALE_VERSION`, `AUCTION_CANCELLED`, `HOST_GRACE_PERIOD_ACTIVE`.
23. **Decide, deliberately, whether/when to split the normalized collections out of `rooms`**. At current (friend-group) scale, keeping auction state embedded is fine as long as every write is atomic and versioned; chat and history are the first candidates to externalize if the room document grows large or contention increases. Don't split preemptively — but don't let the "documented but unused" schema rot either; pick one and note the decision.

F. WebRTC isolation

24. Media stays fully independent of auction state — a peer-connection failure must never block a bid, and a settlement must never interrupt a call. Signaling continues through the dedicated `signals` route, not through MongoDB or the auction snapshot.

G. Observability

25. Structured logs per command: `auctionId`, `commandId`, `participantId`, `teamId`, `currentPlayerId`, `previousVersion`, `newVersion`, `command`, `result`, `latencyMs`. For bids specifically: `BID_RECEIVED` → `BID_VALIDATED` → `BID_COMMITTED` → `BID_EVENT_EMITTED`. For presence: log every heartbeat-miss and every cancellation with its trigger (`explicit_leave` vs `heartbeat_timeout`). Never log credentials or media stream contents.
26. Measure, don't guess: click → request sent → server received → MongoDB committed → event emitted → other clients rendered. Attribute lag to frontend, network, API, MongoDB, transport, or rendering before "fixing" anything.

Phase 3 — Refactor Approach

1. **Audit** (Phase 1 above) — verify what the docs claim about revision numbers and timer epochs is actually enforced everywhere; report discrepancies before changing code.
2. **Design** — confirm state machine, command/event model, timer model, MongoDB concurrency strategy, realtime transport, presence/grace-period policy, chat separation, reconnection strategy. Map each reported symptom to the specific fix that resolves it.
3. **Refactor incrementally** — preserve auth, dashboard, player database, team/profile pages, and WebRTC as-is; touch only the auction-state synchronization, presence/cancellation logic, and chat/event separation.
4. **Test** — typecheck, lint, unit, integration, build, then concurrency-specific tests (below).
5. **Stress test** — 1 host + 4–10 participants: rapid/simultaneous bidding, timer expiry, host actions, chat flood, network delay, duplicate requests, refresh, disconnect/reconnect, host backgrounding tab mid-auction. Nothing should skip a player, sell twice, un-sell a sold player, accept a late bid, reset the timer unexpectedly, advance twice, falsely cancel a live auction, or leave clients permanently diverged.

Required Automated Tests

- **Basic flow:** signup/login → create → join → start → bid → sell → next player → complete → results.
- **Timer:** starts, counts down, expires; unsold with no bid, sold with a bid; verify identical countdown across two simulated clients with an artificial clock offset.
- **Bid race:** bid arriving at/after timer expiry — exactly one settlement.
- **Duplicate command:** same `commandId` sent twice — one effect only.
- **Double sell:** `SELL`, `SELL` — one sale only.
- **Concurrent bids:** multiple users bidding near-simultaneously — one deterministic winner.
- **Chat vs. bid contention:** flood chat while bidding is active — bid latency/version integrity unaffected.
- **Refresh recovery:** mid-bid, mid-overlay, mid-timer — state recovers correctly.
- **Disconnect/reconnect:** participant catches up to current auction *and* chat state.
- **Host transient disconnect:** short heartbeat gap during a live auction — auction is **not** cancelled, resumes normally.
- **Host true disconnect / explicit leave:** auction correctly transitions to `CANCELLED`, results still show accumulated bids/squads.
- **Multi-client convergence:** 1 host + 4 teams + simultaneous bids all converge to identical authoritative state.
- **Invariant assertions**, run continuously:

```
assert(room.version >= previousVersion);
assert(intersection(room.soldPlayerIds, room.unsignedPlayerIds).length === 0);
assert(room.teams.every(team => team.spent <= team.budget));
// plus: no duplicate sale, no duplicate unsigned, no skipped player,
// no cancellation without an explicit trigger reason, no impossible state
```

UX Bar (end-to-end)

- Auth and dashboard feel instant — no polling needed there, just standard request/response.
 - Bid button responds the instant it's clicked; the confirmed amount appears for every other participant without waiting on a polling tick.
 - The countdown moves smoothly while every client stays pinned to the same server deadline.
 - `SOLD` (player, amount, team) appears simultaneously for all participants; the next lot begins on the server's schedule.
 - Chat feels live but never competes with bid-critical traffic for latency.
 - A host's brief network hiccup is invisible to participants — no false "auction cancelled."
 - Reconnecting after a real drop restores the exact current state — bid, timer, chat — without a manual refresh.
-

Definition of Done

- ☐ Server is the sole authority for auction status, timer expiry, and settlement.
- ☐ Every state-mutating write (bid, sell, unsigned, advance, start, join, leave, cancel, chat) goes through the same version-checked path — verified, not assumed.

- ☐ Timer is timestamp-derived and rendered client-side from a synced server deadline; no client-driven progression.
- ☐ Settlement (`settleCurrentLot`) is idempotent regardless of trigger source.
- ☐ Duplicate commands are safe via idempotency keys.
- ☐ Stale responses can never overwrite newer client state (version guard confirmed on every path).
- ☐ Realtime events (not 1s polling) carry live auction updates; polling remains only for hydration/recovery.
- ☐ Chat has its own event stream/versioning and cannot delay or race auction-critical writes.
- ☐ Host presence has a grace period; transient disconnects never auto-cancel a live auction; explicit leave and heartbeat-timeout are logged and handled distinctly.
- ☐ Reconnection restores auction *and* chat state automatically.
- ☐ Players cannot be skipped, double-sold, or incorrectly unsold; teams cannot overspend.
- ☐ WebRTC remains fully isolated from auction state.
- ☐ Concurrency/race-condition tests pass; typecheck, lint, and build all pass.
- ☐ No client-side code can override server-authoritative fields directly.

Mental Model

```

CLIENTS SEND COMMANDS
    ↓
SERVER VALIDATES (incl. version + idempotency key)
    ↓
SERVER ATOMICALLY CHANGES STATE
    ↓
SERVER CREATES EVENT (auction or chat, distinct streams)
    ↓
EVENT BROADCAST TO ALL CLIENTS (realtime, polling as fallback)
    ↓
ALL CLIENTS CONVERGE
  
```

Timer: `SERVER DEADLINE` → `CLIENT CLOCK SYNC` → `SMOOTH LOCAL RENDERING`.

Settlement: `TIMER EXPIRES` / `HOST SETTLES` → `SERVER LOCK` / `CAS` → `SETTLE LOT` (idempotent) → `RECORD RESULT` → `INCREMENT VERSION` → `EMIT EVENT` → `START NEXT LOT`.

Presence: `HEARTBEAT MISS` → `GRACE WINDOW` → (`RECONNECT: resume normally`) OR (`TIMEOUT: CANCELLED, clearly logged and labeled`).

Priority order: correctness first, then latency, then UI polish. Keep the auth/dashboard/player-database/profile surfaces as they are — the redesign is scoped to the live-room, presence, and chat layers where the real-time guarantees actually matter.